

Архитектурное описание системы

1. Общие сведения о системе

Разрабатываемая система представляет собой однопользовательскую игру в жанре roguelike с консольным интерфейсом. Игрок управляет персонажем с клавиатуры, перемещается по карте уровня, взаимодействует с мобами и предметами. Предметы могут находиться на карте, подбираться в инвентарь, а также надеваться и сниматься, надетые предметы влияют на характеристики персонажа. Игра состоит из последовательности уровней, уровни могут быть заранее определёнными (загружаться из набора уровней) или генерироваться процедурно. Игра завершается победой при прохождении всех уровней либо поражением при смерти персонажа.

2. Architectural drivers

При проектировании системы учитываются следующие факторы. Архитектура должна быть расширяемой, так как дальнейшие задания предполагают развитие механик и усложнение логики. Требуется высокая скорость разработки, компоненты должны быть слабо связаны, чтобы изменения в генерации уровня, данных или рендере не приводили к переработке всего приложения. Интерфейс должен работать без заметных задержек при вводе и перерисовке. Дополнительным ограничением является простота реализации и понятность структуры для учебного проекта.

3. Роли и случаи использования

В системе присутствует один основной актор - игрок. Игрок запускает приложение и взаимодействует с игрой через клавиатуру. Система отображает состояние уровня и результаты действий пользователя.

3.1. Прецедент П1. Игра в Roguelike игру

Основной исполнитель: игрок.

Заинтересованные лица и их требования: игрок хочет получить удовольствие от игры и проникнуться духом старых игр.

Основной успешный сценарий.

1. Игрок запускает игру.
2. Система генерирует (или загружает) первый уровень.
3. Пользователь проходит уровень, уничтожая всех мобов.
4. Система предлагает пользователю перейти на новый уровень.
5. Пользователь подтверждает переход.

Действия, описанные в пп. 2-5, повторяются, пока пользователь не пройдет все уровни.

6. Система поздравляет пользователя с прохождением игры.

Расширения.

*а. При закрытии пользователем терминала с игрой.

1. Приложение завершается, не делая при этом никаких дополнительных действий.

4. Диаграмма компонентов и описание компонентов

Система разделена на набор основных компонентов. Компонент генерации событий ответственен за формирование событий, на которые система должна реагировать. Например, он может слушать нажатия клавиш и создавать соответствующие события, либо генерировать события с заданной периодичностью.

Компонент ядра ответственен за обработку событий, обновление состояния игры и вызов перерисовки интерфейса. Интерфейс занимается отрисовкой текущего состояния системы и инкапсулирует выбранную UI-библиотеку.

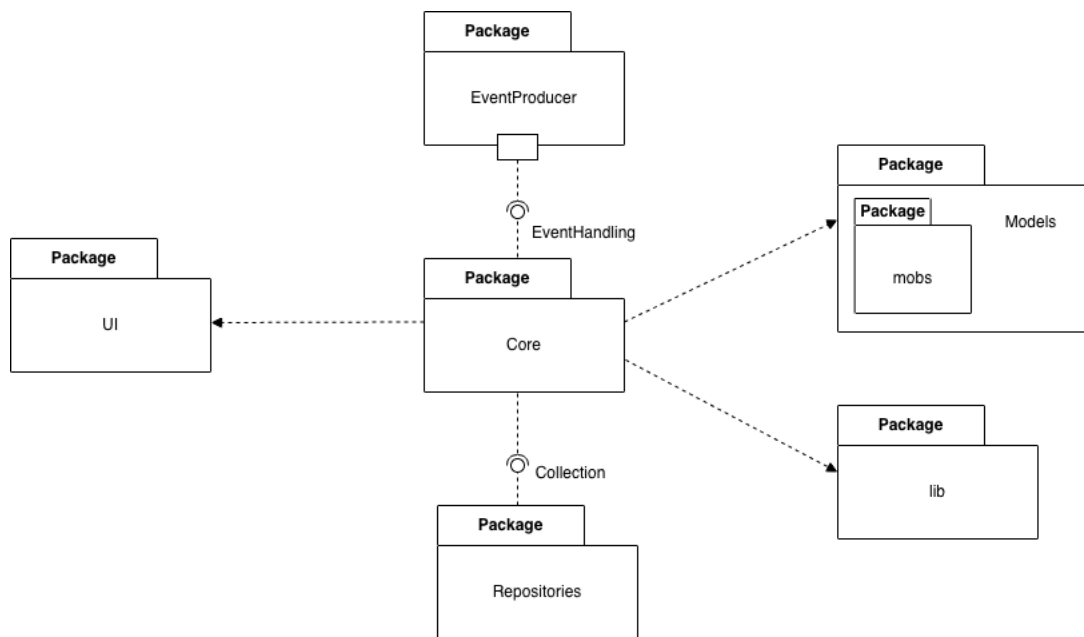


Рис. 1. Диаграмма компонентов (пакетов)

Компонент доменной модели представляет набор сущностей предметной области, используемых другими компонентами. Отдельно выделены вспомогательные алгоритмы (например, генерация уровней) и репозитории, содержащие конфигурируемые параметры игры (набор уровней, параметры генерации и прочие данные).

5. Диаграмма классов и логическая структура

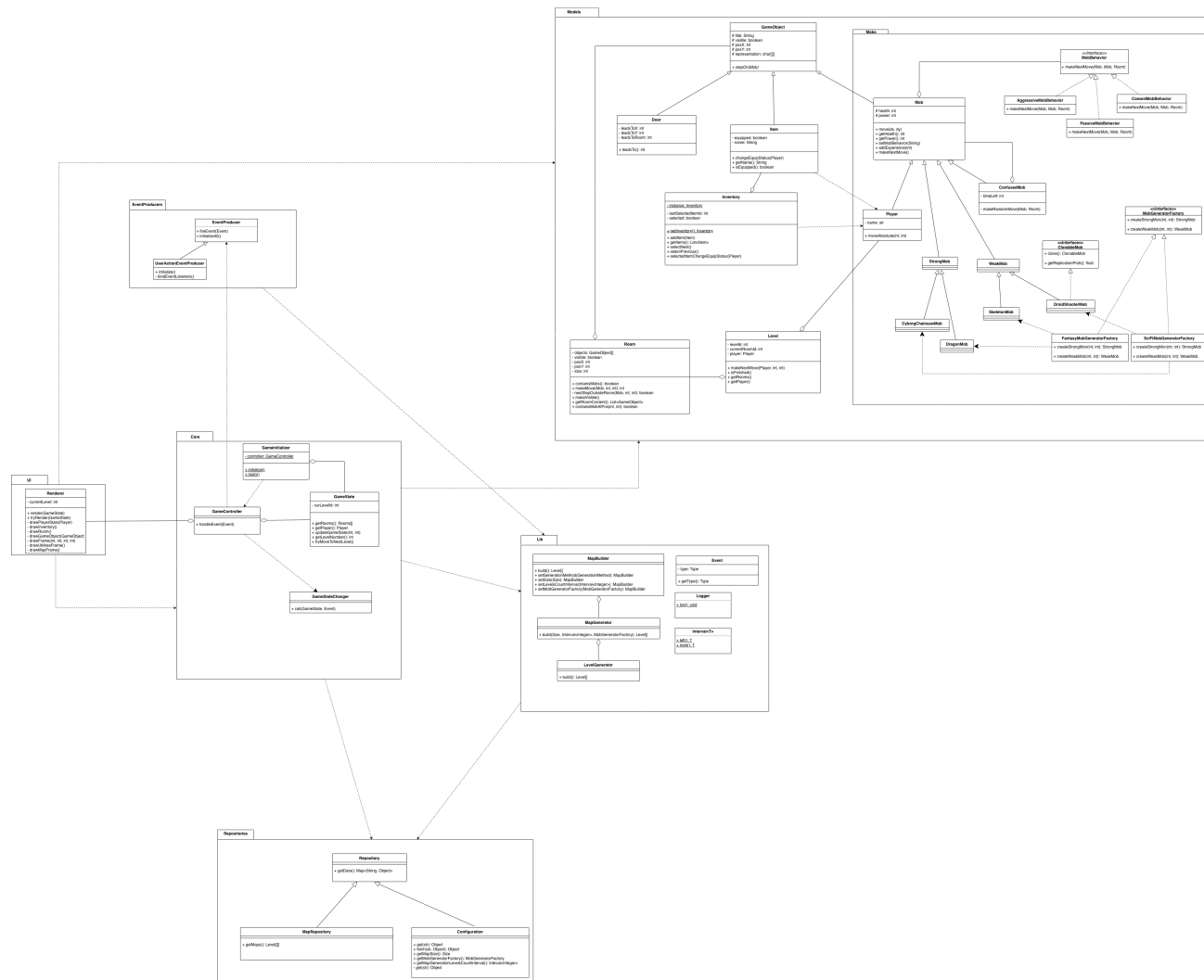


Рис. 2. Диаграмма классов приложения

Подсистема генерации событий формирует события на основе действий пользователя и передает их в ядро для обработки. В ядре располагаются контроллер обработки событий и объект состояния игры. При получении события состояние обновляется, после чего вызывается перерисовка интерфейса.

Доменная модель содержит базовый класс игрового объекта с координатами и отображаемым символом. Объекты уровня (предметы, mobs, переходы) определяют поведение при взаимодействии, например при наступании на клетку.

Поведение мобов реализовано через паттерн «Стратегия»: выделен интерфейс поведения моба и несколько реализаций для разных типов поведения (например, агрессивное, пассивное и трусливое). Дополнительно реализован временный эффект изменения поведения (оглушение/путаница) через паттерн «Декоратор», когда объект-обертка над мобом изменяет его логику на заданное время.

Для улучшения генерации карты используется паттерн «Строитель». Класс-строитель позволяет параметризовать генератор: задать, генерировать карту алгоритмически или загрузить из файла, указать размеры карты, количество уровней, а также передать фабрику мобов. После настройки вызывается метод `build()`, возвращающий готовую карту/уровень.

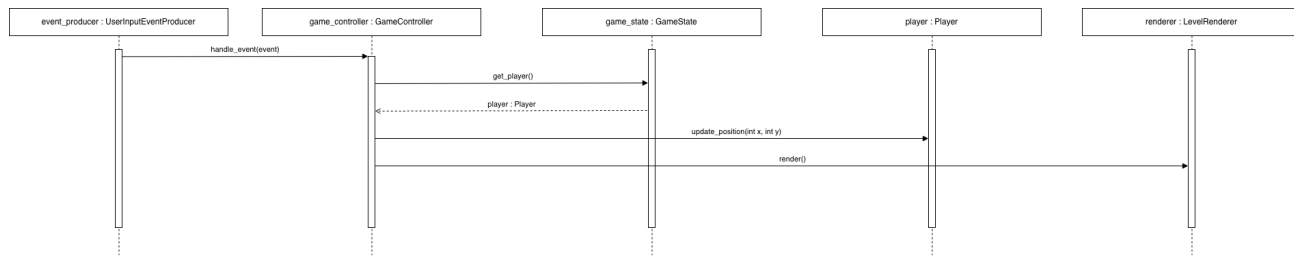
Для поддержки разных «стилей» мобов используется паттерн «Абстрактная фабрика». Предусмотрен интерфейс фабрики мобов и две конкретных реализации (например, фэнтези и научная фантастика). Каждая фабрика создает согласованный набор мобов с соответствующими характеристиками и представлением. Фабрика передается в строителя карты и используется при заполнении карты мобами.

Для мобов, способных к репликации на поле боя, используется паттерн «Прототип». Такие мобы реализуют интерфейс клонирования и могут порождать копии самих себя (например, каждый ход с вероятностью p создается клон в соседней свободной клетке). Это позволяет создавать новые экземпляры без знания конкретного типа моба в месте репликации.

6. Диаграммы последовательностей (взаимодействия)

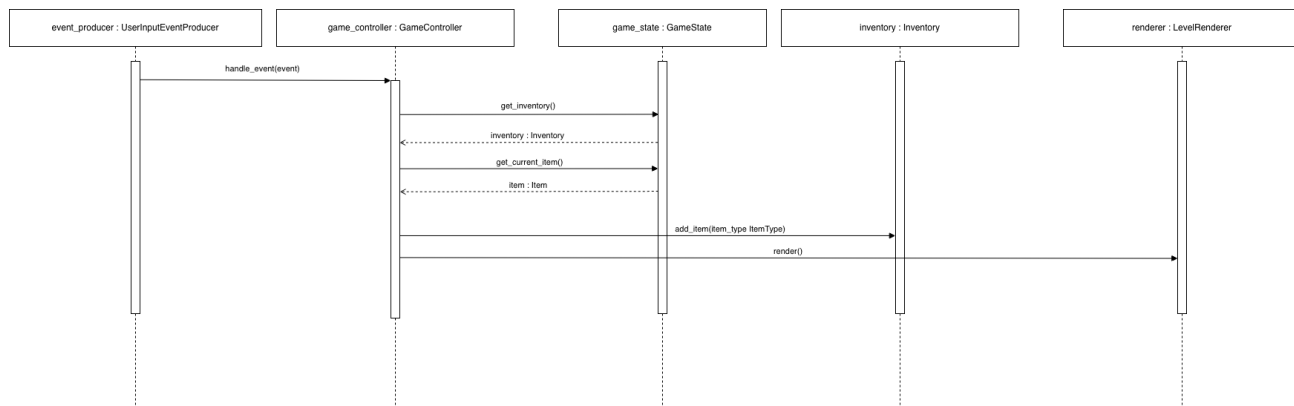
Диаграммы последовательностей фиксируют типовые сценарии работы системы и показывают передачу управления между объектами при обработке событий пользователя.

6.1. Перемещение персонажа



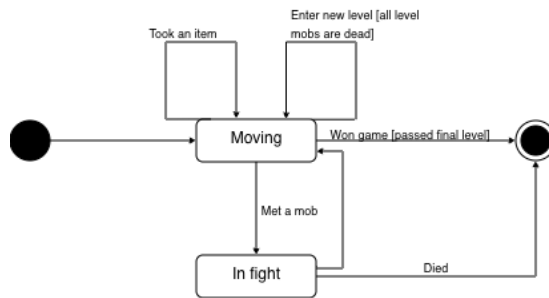
`UserActionEventProducer` формирует событие и инициирует его обработку в `GameController.handle_event(Event)`. `GameController` получает игрока через `GameState.get_player(): Player` и изменяет позицию игрока (например, через метод `Player.update_position(int x, int y)` или эквивалентный вызов). После изменения состояния `GameController` вызывает `Renderer.render(GameState)`.

6.2. Подбор предмета



`UserActionEventProducer` формирует событие подбора предмета и передаёт его в `GameController.handle_event(Event)`. `GameController` получает доступ к `Inventory` (через `GameState.inventory` или через аксессор) и добавляет предмет в инвентарь (например, вызовом `Inventory.add_item(...)`). Далее выполняется `Renderer.render(GameState)`.

7. Диаграмма состояний



Состояния игры представлены режимами Moving (исследование уровня), In fight (бой) и Game over (завершение игры). Переход Moving > In fight происходит при встрече с мобом. После боя возможен переход в Moving при победе или переход в Game over при смерти игрока. Переход на следующий уровень выполняется при выполнении условия завершения уровня и не меняет основной режим (остается Moving), но обновляет текущий Level. При прохождении последнего уровня выполняется переход в Game over с результатом “победа”.